PART 1 (Continued): ADVANCED ERROR DETECTION METHODS

3. Checksum Calculation — TB Pages 58–59

A checksum is a mathematical value calculated from a block of data before transmission.

1. The sender computes a numerical sum of all bytes in the data block using a predetermined algorithm.
2. If the sum exceeds 255 (1 byte), modulo 256 is applied ($X = \text{Sum } \% \text{ } 256$) and subtracted ($255 - X$) to determine the checksum value.
3. The checksum is appended to the block and transmitted.
4. Upon arrival, the recipient re-calculates the checksum on the received payload and compares it against the received checksum byte.
5. If the values match, transmission is accepted; if they disagree, an error occurred and a re-transmission request is issued.

4. Echo Checks — TB Pages 59–60

In an **Echo Check**, when data is sent to the recipient, the recipient immediately sends a copy of the received data back (echoes it) to the sender. The sender compares the echoed data with the original master copy.

- **Advantage:** Very simple logic; requires minimal processing power.
- **Critical Limitations:**
  - It doubles network bandwidth consumption (every packet is transmitted twice).
  - If a mismatch occurs, the sender cannot tell whether the error occurred during the original transmission to the recipient or during the return echo journey.

5. Check Digits (ISBN-13 & Modulo 11) — TB Pages 60–61

While parity and checksums detect *data transmission* errors, **Check Digits** are designed to detect human data entry errors (e.g. entering a barcode or account number).

Common human entry errors detected by Check Digits:

- **Transcription Error:** Typing an incorrect digit (e.g. typing `5389` instead of `5379`).
- **Transposition Error:** Swapping two adjacent digits (e.g. typing `5398` instead of `5389`).
- **Omission / Extra Digit Error:** Missing a digit or entering an extra digit.
- **Phonetic Error:** Mishearing numbers (e.g. typing `13` instead of `30`).

> **ISBN-13 Algorithm:**
> `Multiply alternating digits by 1 and 3:`
> `Sum = (d1 × 1) + (d2 × 3) + (d3 × 1) + (d4 × 3) + ... + (d12 × 3)`
> `Remainder = Sum % 10`
> `Check Digit = (Remainder == 0) ? 0 : (10 - Remainder)`